4CSPL1012: PROBLEM SOLVING USING PYTHON

Module 2: Basics of Python – Part I

Dr. Pandava Sudharshan Babu & Python Faculty Team

School of Engineering and Technology (SoET)

Credits: Respective owners

Codes: Click on this link to get Google colab file

Variables & Data Types:

- A **variable** is a name given to a memory location used to store data.
- Computer memory has many storage locations
- Each variable points to a location
- That location stores a value

Example:

age = 20
Variable name → age
Value stored → 20

Dynamic Typing in Python

- Python automatically assigns data type based on value.
- “You don’t declare type, Python decides it automatically”

Example:

x = 10 → integer

x = "Hello" → string

👉 Same variable, different types!

Basic Data Types in Python:

Data Type	Description	Example
int	Whole numbers	10, 25
float	Decimal numbers	10.5, 99.9
string	Text data	"Hello"
boolean	True/False	True, False

Real-World Examples

Scenario	Variable	Data Type
Age of student	age	int
Salary	salary	float
Name	name	string
Login status	is_logged_in	boolean

Identify Data Types

Input	Expected Type
25	?
99.5	?
"Python"	?
True	?
"123"	? (Tricky!)

Answers:
25 → int
99.5 → float
"Python" → string
True → boolean
"123" → string

Real-Life Activity

Take the below scenarios:

- Bank account → balance, account_holder
- College → student_name, marks, attendance
- E-commerce → product_name, price, availability

Now

✓ Identify variables

✓ Assign data types

Identifiers

Identifiers are the names that identify the elements such as variables and functions in a program.

Rules for Valid Identifiers:

1. Identifiers can only contain letters (a-z, A-Z), digits (0-9), and underscores (_).
2. Identifiers must start with a letter (a-z, A-Z) or an underscore (_).
3. Identifiers are case-sensitive (name and Name are different).
4. Python keywords (e.g., if, for, class) cannot be used as identifiers.

- *Keywords*, also called *reserved words*, have special meanings in Python.
- For example, **import** is a keyword, which tells the Python interpreter to import a module to the program.

Identifier	Valid/Invalid	Reason
`name`	Valid	Follows all rules.
`age2`	Valid	Starts with a letter, contains a digit.
`_height`	Valid	Starts with an underscore.
`first_name`	Valid	Uses underscores, no invalid characters.
`PI`	Valid	All uppercase letters, no issues.
`2nd_place`	Invalid	Starts with a digit.
`student-name`	Invalid	Contains a hyphen (`-`), not allowed.
`first@name`	Invalid	Contains `@`, which is a special character.
`class`	Invalid	`class` is a Python keyword.
`student name`	Invalid	Contains a space, which is not allowed.
`$amount`	Invalid	Contains `$`, which is a special character.

Python Keywords

The following keywords are reserved by the Python language. They should not be used for anything other than their predefined purpose in Python.

and	else	in	return
as	except	is	True
assert	False	lambda	try
break	finally	None	while
class	for	nonlocal	with
continue	from	not	yield
def	global	or	
del	if	pass	
elif	import	raise	

Which of the following identifiers are valid? Which are Python keywords

miles, **Test**, **a+b**, **b–a**, **4#R**, **$4**, **#44**, **apps**

if, **elif**, **x**, **y**, **radius**

Note:

case sensitive

Note

Because Python is case sensitive, `area`, `Area`, and `AREA` are all different identifiers.

descriptive names

Tip

Descriptive identifiers make programs easy to read. Avoid using abbreviations for identifiers. Using complete words is more descriptive. For example, `numberOfStudents` is better than `numStuds`, `numOfStuds`, or `numOfStudents`. We use descriptive names for complete programs in the text. However, we will occasionally use variables names such as `i`, `j`, `k`, `x`, and `y` in the code snippets for brevity. These names also provide a generic tone to the code snippets.

variable naming convention

camelCase

Tip

Use lowercase letters for variable names, as in `radius` and `area`. If a name consists of several words, concatenate them into one, making the first word lowercase and capitalizing the first letter of each subsequent word—for example, `numberOfStudents`. This naming style is known as the *camelCase* because the uppercase characters in the name resemble a camel's humps.

Breakdown of keywords:

1. Boolean Values:
   - `True`, `False`, `None`
2. Logical Operators:
   - `and`, `or`, `not`, `is`, `in`
3. Control Flow:
   - `if`, `elif`, `else`
   - `for`, `while`, `break`, `continue`
   - `try`, `except`, `finally`, `raise`
4. Function & Class Definition:
   - `def`, `return`, `yield`
   - `class`
5. Exception Handling:
   - `try`, `except`, `finally`, `raise`
6. Miscellaneous:
   - `as`, `assert`, `del`, `global`, `import`, `lambda`, `nonlocal`, `pass`, `with`
7. Asynchronous Programming:
   - `async`, `await`

Different versions of print in Python:

1. Basic print()

```python
print("Hello CMR University")
```

Output:

```
Hello CMR University
```

2. Printing Multiple Items:

- You can print multiple items separated by commas, and they will be separated by a space by default.

```python
print("I have", 5, "Apples", 15, "Bananas")
```

Output:

```
I have 5 Apples 15 Bananas
```

3. String Concatenation with +

- You can concatenate strings using the + operator, but all items must be strings.

```
name = "Welcome to Python crash course"
print("Hello, " + name + "!")
```

Output: `Hello, Welcome to Python crash course!`

4. sep Parameter:

- The sep parameter controls how multiple arguments are separated. By default, it is a space (' '), but you can change it.

```
print("apple", "banana", "Mango", sep=", ")
```

Output: `apple, banana, Mango`

5. end Parameter

- The end parameter defines what is printed at the end of the print statement. By default, it is a newline (\n), but you can change it to something else.

Example 1:

```
print("Hello", end=" - ")
print("SOET")
```

Output: Hello - SOET

Example 2:

```
print("Hello", end=" $ ")
print("SOET")
```

Output: Hello $ SOET

6. Formatted String (f-strings):

- Introduced in Python 3.6, f-strings allow variables to be embedded directly into string literals.

Example1:

```python
name = "Alice"
age = 25
city = "Bangalore"

print(f"My name is {name}, I am {age} years old, and I live in {city}.")
```

Output:

```
My name is Alice, I am 25 years old, and I live in Bangalore.
```

7. Printing with Escape Characters: You can use special escape sequences like \n (newline), \t (tab), and \\ (backslash).

Example

```python
print("Hello\nSOET")    # Newline
print("A\tB\tC")         # Tab space
print("This is a backslash: \\")  # Backslash
```

Output:

```
Hello
SOET
A    B    C
This is a backslash: \
```

Note:

Escape Sequence	Purpose	Example Output
`\n`	Moves to next line	Hello SOET
`\t`	Gives tab space	A B C
`\\`	Prints single backslash	\

8. Printing Without Newline (end='')

By default, print() adds a newline at the end. You can change this by setting end="".

Example:

```
print("Hello", end=" ")
print("World")
```

Output:

```
Hello World
```

Reading Input from the Console

- Programs often need **input from the user** to perform tasks.
- "Input allows interaction between user and program"

Using `input()` Function:

Syntax:

```
variable = input("Enter a value: ")
```

Example:

```
name = input("Enter your name: ")
print("Hello,", name)
```

Output:

```
Enter your name: CMR University
Hello, CMR University
```

⚠️ Important Point

➢ 👉 By default, input() always returns **string data type**

Example:

```
age = input("Enter age: ")
```

- Even if user enters 25 → it is stored as "25" (string)

Converting Input to Required Type:

- The solution is to use type conversion functions:

```
age = int(input("Enter age: "))
salary = float(input("Enter salary: "))
```

Purpose	Function	Example
Integer input	`int()`	`int(input())`
Decimal input	`float()`	`float(input())`
String input	`input()`	`input()`

Example Program

```python
name = input("Enter your name: ")
age = int(input("Enter your age: "))

print("Name:", name)
print("Age:", age)
```

Using eval() in Python

- eval() evaluates a string as a Python expression.

Use Case	Example	Result
Float conversion	`eval("34.5")`	34.5
Integer conversion	`eval("100")`	100
Expression	`eval("5+2")`	7
Complex expression	`eval("2*(3+4)")`	14

Using eval() with input():

- input() Always returns **string**
- 👉 eval(input()) - Converts input into **actual data type or expression result**

Example:

```python
value = eval(input("Enter a number: "))
print(value)
print(type(value))
```

Output:

```
Enter a number: 143
143
<class 'int'>
```

Example:

```python
value = eval(input("Enter a decimal number: "))
print(value)
print(type(value))
```

Output:

```
Enter a decimal number: 10.5
10.5
<class 'float'>
```

Summary

Input Given by User	Using `input()`	Using `eval(input())`	Result Type
`25`	`"25"`	`25`	int
`10.5`	`"10.5"`	`10.5`	float
`"Hello"`	`"Hello"`	`"Hello"`	string
`True`	`"True"`	`True`	boolean
`5 + 2`	`"5 + 2"`	`7`	int
`3 * 4.5`	`"3 * 4.5"`	`13.5`	float

Note:

- input() → always returns **string**
- eval(input()) → **evaluates and converts** input

"eval(input()) converts and executes user input as a Python expression."

Basic Programs:

1. Area of Circle

```python
# Assign value to radius
radius = eval(input("Enter the value of radius: "))

# Compute area
area = radius * radius * 3.14159

# Display result
print("The area of the circle with radius", radius, "is", area)
```

Output:

```
Enter the value of radius: 5
The area of the circle with radius 5 is 78.53975
```

Note: # is used for comments in Python

2. Area of Rectangle

```python
# Input length and breadth
length = eval(input("Enter length: "))
breadth = eval(input("Enter breadth: "))

# Compute area
area = length * breadth

# Output
print("Area of rectangle is", area)
```

Output:

```
Enter length: 3
Enter breadth: 5
Area of rectangle is 15
```

Program 3: Simple Interest

```python
# Input values
principal = eval(input("Enter principal amount: "))
rate = eval(input("Enter rate of interest: "))
time = eval(input("Enter time: "))

# Compute simple interest
si = (principal * rate * time) / 100

# Output
print("Simple Interest is", si)
```

Output:

```
Enter principal amount: 10000
Enter rate of interest: 12
Enter time: 4
Simple Interest is 4800.0
```

Program 4: Average of 3 Numbers

```python
# Input numbers
a = eval(input("Enter first number: "))
b = eval(input("Enter second number: "))
c = eval(input("Enter third number: "))

# Compute average
avg = (a + b + c) / 3

# Output
print("Average is", avg)
```

Output:

```
Enter first number: 4
Enter second number: 6
Enter third number: 3
Average is 4.333333333333333
```

Practice Programs:

- ✓ Program 5: Swap Two Numbers
- ✓ Program 6: Temperature Conversion (Celsius to Fahrenheit)
- ✓ Program 7: Square and Cube of a Number
- ✓ Program 8: Calculate Discounted Price
- ✓ Program 9: Sum of Two Numbers
- ✓ Program 10: Perimeter of Rectangle

Solutions:

✓ Program 5: Swap Two Numbers

```python
# Input values
a = eval(input("Enter first number: "))
b = eval(input("Enter second number: "))

# Swap
a, b = b, a

# Output
print("After swapping:")
print("a =", a)
print("b =", b)
```

✓ Program 6: Temperature Conversion (Celsius to Fahrenheit)

```python
# Input temperature
celsius = eval(input("Enter temperature in Celsius: "))

# Convert
fahrenheit = (celsius * 9/5) + 32

# Output
print("Temperature in Fahrenheit is", fahrenheit)
```

✓ Program 7: Square and Cube of a Number

```python
# Input number
num = eval(input("Enter a number: "))

# Compute
square = num * num
cube = num * num * num

# Output
print("Square =", square)
print("Cube =", cube)
```

✓ Program 8: Calculate Discounted Price

```python
# Input price and discount percentage
price = eval(input("Enter original price: "))
discount = eval(input("Enter discount percentage: "))

# Compute discount
discount_amount = (price * discount) / 100
final_price = price - discount_amount

# Output
print("Discount amount is", discount_amount)
print("Final price is", final_price)
```

✓ Program 9: Sum of Two Numbers

```python
# Input numbers
a = eval(input("Enter first number: "))
b = eval(input("Enter second number: "))

# Compute sum
sum_result = a + b

# Output
print("Sum is", sum_result)
```

✓ Program 10: Perimeter of Rectangle

```python
# Input length and breadth
length = eval(input("Enter length: "))
breadth = eval(input("Enter breadth: "))

# Compute perimeter
perimeter = 2 * (length + breadth)

# Output
print("Perimeter of rectangle is", perimeter)
```